Introduction to Systems Engineering

Course: AEZC517

Generated: 2026-01-11 13:54

Warning

This handout contains unreviewed AI-generated questions. Questions should be reviewed before distribution to students.

Table of Contents

Session 1: Introduction to Systems Engineering ... Instructor-Led Story Framework for PPT Preparation ... 🎯 Session Objectives (For Instructor) ... 🔗 Connection to Prior Knowledge ... 📖 INSTRUCTOR-LED STORY ARC ... 📊 PPT DESIGN GUIDANCE ... 🎯 INSTRUCTOR NOTES ... ✅ Session Success Criteria ... 🔗 Bridge to Session 2 ...

Session 1: Introduction to Systems Engineering

Instructor-Led Story Framework for PPT Preparation

Duration: 120 minutes (90 minutes lecture + 30 minutes discussion/Q&A) **Approach:** Story-Based with Real-World Failure Case Studies **PPT Target:** 28-30 slides

---

🎯 Session Objectives (For Instructor)

By the end of this session, students will be able to: - Understand the critical importance of systems engineering through real-world failure examples - Recognize the cost and consequences of poor systems engineering - Define what a system is and identify key system characteristics - Articulate the value proposition of systems engineering in automotive contexts - Begin developing a systems thinking mindset

Learning Outcomes Addressed: - **LO1:** Describe processes, methods, and practices of systems engineering (Understand) - **LO3:** Recognize important systems engineering and systems thinking strategies and practices (Understand)

🔗 Connection to Prior Knowledge

This is Session 1, so we're building from: - General engineering education and principles - Students' experiences with complex products and systems - Basic understanding of engineering failures from news/media - Undergraduate engineering fundamentals (requirements, design, testing)

Bridge: "You've learned to design components. Now we learn to design systems."

📖 INSTRUCTOR-LED STORY ARC

SETUP: The Wake-Up Call (Slides 1-8, ~20 minutes)

Narrative Voice: Direct, engaging, building urgency about why systems engineering matters

---

Slide 1: Title Slide

Visual: Professional title slide with modern automotive imagery (connected vehicle, complex system diagram overlay)

Instructor Narration:

> "Welcome to Automotive Systems Engineering. This course is going to change how you think about engineering. Not what you engineer, but *how* you think about it. And we're going to start with some failures. Spectacular, expensive, deadly failures. Because that's where the most important lessons live."

PPT Content:

```
AUTOMOTIVE SYSTEMS ENGINEERING
AE ZG517 / AEL ZG517
Session 1: Introduction to Systems Engineering

Instructor: Sajeeth Kumar
Duration: 120 minutes
```

Learning Outcomes: Setting context for LO1, LO3

Slide 2: A Question to Start

Visual: Image of a modern vehicle dashboard with hundreds of warning indicators and sensors

Instructor Narration:

> "Look at this dashboard. Fifty years ago, a car had maybe five gauges. Today? This vehicle has over 100 million lines of code, more than 150 electronic control units, thousands of components, and they all have to work together. Here's my question for you: **How do we make sure all these pieces work together without killing someone?**
>
> That's not dramatic exaggeration. That's the literal challenge of systems engineering. And when we get it wrong, people die."

PPT Content:

```
Modern Automotive Complexity:
• 100+ million lines of code
• 150+ Electronic Control Units (ECUs)
• 30,000+ components
• Multiple suppliers, teams, disciplines
```

```
• Safety-critical requirements

THE QUESTION: How do we make it all work together?
```

Learning Outcomes: LO3 - Recognizing the need for systems engineering

---

Slide 3: Case Study Preview - Three Disasters

Visual: Three images side by side: Apollo 13 spacecraft, Therac-25 medical device, Toyota vehicle

Instructor Narration:

> "Today, I'm going to tell you three stories. Three very different systems. Three catastrophic failures. And all three have the same root cause: failures in systems engineering.
>
> • **Apollo 13:** Lost in space, 200,000 miles from Earth
> • **Therac-25:** Six patients killed by radiation overdoses
> • **Toyota Unintended Acceleration:** 89 deaths, 9 million vehicles recalled
>
> These aren't just history lessons. Each one changed how we practice engineering. Each one has lessons for automotive engineers today."

PPT Content:

```
Three Failure Case Studies:
1. Apollo 13 (1970) - Space exploration
2. Therac-25 (1985-1987) - Medical device
3. Toyota Recalls (2009-2011) - Automotive

Common Thread: Systems Engineering Failures
```

Learning Outcomes: LO3 - Recognizing systems engineering failures

---

Slide 4: Why Start with Failures?

Visual: Quote on screen with engineering background imagery

Instructor Narration:

> "You might wonder why we're starting a course with disasters. Here's why: Success is invisible. When systems engineering works, everything just... works. Nobody notices. But failures? Failures teach us what happens when we ignore systems engineering principles.
>
> As Henry Petroski wrote, 'Success is foreseeing failure.' That's what we're doing today. We're going to study failure so you can foresee it, prevent it, and succeed where others failed."

PPT Content:

```
"Success is foreseeing failure."
- Henry Petroski

Why Study Failures?
✓ Failures reveal what we miss
✓ Success is invisible; failure is instructive
✓ Learn from others' mistakes
✓ Understand consequences of poor systems engineering
✓ Build intuition for systems thinking
```

Learning Outcomes: LO3 - Understanding systems engineering practices

---

Slide 5: Your Engineering Context

Visual: Split image - traditional component engineering vs. systems engineering approach

Instructor Narration:

> "Most of your engineering education so far has focused on **components**: design this gear, analyze this circuit, optimize this algorithm. And that's important. But in the real world, you're never building just a component. You're building a **system**.
>
> A component works in isolation. A system works in context. And that context changes everything. Today, we begin shifting your mindset from component engineer to systems engineer."

PPT Content:

```
Component Engineering vs. Systems Engineering

COMPONENT FOCUS:
• Design individual parts
• Optimize single functions
• Controlled conditions
• Known interfaces

SYSTEMS FOCUS:
• Integrate multiple parts
• Balance competing requirements
• Operating in real environments
• Emergent behaviors and interactions

→ Both are necessary. Systems engineering provides the framework.
```

Learning Outcomes: LO1, LO3 - Understanding systems engineering approach

---

Slide 6: Today's Learning Journey

Visual: Flow diagram showing session structure: Failures → Why SE Matters → What is a System → Systems Thinking

Instructor Narration:

"Here's our path today. We start with three failure stories that illustrate what goes wrong. Then we extract the lessons—why systems engineering matters. From there, we'll build the foundation: what is a system, what are its characteristics, and how do we think about systems differently.

By the end, you won't just understand *that* systems engineering matters. You'll understand *why*, and you'll begin developing the mindset to practice it."

PPT Content:

```
Today's Journey:

1. Failures (20 min) - Three case studies
2. Consequences (15 min) - Cost of poor SE
3. Foundations (25 min) - What is a system?
4. Systems Thinking (20 min) - New mindset
5. Application (10 min) - Automotive context

Goal: Understand WHY systems engineering exists
```

Learning Outcomes: LO1, LO3

Slide 7: Ground Rules for Case Studies

Visual: Simple text slide with guiding principles

Instructor Narration:

"Before we dive into the failures, three ground rules:

First, we're not here to blame individuals. These failures involved brilliant engineers. The problem was systemic, not personal.

Second, ask yourself during each story: 'Could this happen in automotive engineering?' The answer is almost always yes.

Third, focus on the *how* and *why*, not just the *what*. We want to understand the systems engineering failures, not just the technical ones.

Ready? Let's start with Apollo 13."

PPT Content:

```
Case Study Ground Rules:

1. No blame - Systemic failures, not personal
2. Automotive relevance - Ask "Could this happen here?"
3. Focus on SE process - Not just technical details

Questions to Consider:
• What systems engineering practices were missing?
• What assumptions failed?
• How could this have been prevented?
```

Learning Outcomes: LO3 - Framework for analyzing SE failures

Slide 8: The Cost of Failure (Teaser)

Visual: Dramatic numbers on screen showing costs

Instructor Narration:

> "One last thought before we begin. The Toyota recall? **$3 billion in direct costs**. Lost reputation? Incalculable. Lives lost? 89 confirmed deaths.
>
> Systems engineering is not an academic exercise. It's not bureaucracy. It's the difference between products that work and products that kill. Between companies that thrive and companies that collapse.
>
> Let's examine how it all goes wrong."

PPT Content:

```
The Stakes:

Toyota Unintended Acceleration:
• $3 billion in direct costs
• 9 million vehicles recalled
• 89 confirmed deaths
• Brand reputation severely damaged

SYSTEMS ENGINEERING IS NOT OPTIONAL.
```

Learning Outcomes: LO3 - Understanding criticality of SE

TRIGGER: The Disasters (Slides 9-16, ~25 minutes)

Narrative Voice: Storytelling mode - vivid, detailed, building tension

---

Slide 9: Case Study 1 - Apollo 13: "Houston, We've Had a Problem"

Visual: Iconic image of Apollo 13 spacecraft, damaged service module

Instructor Narration:

> "April 13, 1970. Three astronauts are two days into their journey to the moon. 200,000 miles from Earth. Then, at 55 hours into the mission, they hear a bang. Oxygen tank 2 has exploded.
>
> The explosion disables the Command Module's power, water, and oxygen. Three men, hurtling through space, in a crippled spacecraft. This is not just an engineering failure. This is a systems engineering failure that began years before launch."

PPT Content:

```
APOLLO 13 OXYGEN TANK EXPLOSION
April 13, 1970 | 55 hours into mission

THE INCIDENT:
• Oxygen tank 2 explodes
• Command Module disabled
• 3 astronauts, 200,000 miles from Earth
• Need to survive 4 days on backup systems

Critical Question: How did this happen?
```

Learning Outcomes: LO3 - Case study introduction

Slide 10: Apollo 13 - The Systems Engineering Failure

Visual: Timeline diagram showing the cascade of decisions

Instructor Narration:

> "Here's what happened. During pre-flight testing, ground crews couldn't remove oxygen from tank 2. So they ran the heater to boil it off. But here's the problem: the thermostatic switches were rated for 28 volts. The ground equipment used 65 volts.
>
> The switches welded shut. Nobody noticed. During the mission, the heater cycled on and off... but the switches couldn't open. The temperature soared. Teflon insulation ignited in the pure oxygen environment. Boom.
>
> Now, was this a design failure? Yes. But more importantly, it was a **systems engineering failure**. Let me show you how."

PPT Content:

```
THE SYSTEMS ENGINEERING FAILURES:

1. Interface Mismatch
   • Tank designed for 28V spacecraft systems
   • Ground equipment operated at 65V
   • No verification of compatibility

2. Design Change Propagation
   • Upgraded to 65V spacecraft systems
   • Did NOT upgrade tank components
   • Change impact analysis failed

3. Testing Limitations
   • Ground test anomaly (couldn't remove O2)
   • Workaround implemented (heat it out)
   • Nobody recognized danger signal

4. Requirements Gap
   • No requirement for ground equipment compatibility
   • No verification of thermostatic switch ratings
```

Learning Outcomes: LO1, LO3 - Understanding SE process failures

---

Slide 11: Apollo 13 - The Systems Lesson

Visual: Diagram showing interconnected system elements that failed

Instructor Narration:

> "Here's the systems engineering lesson: **Every component exists in a context**. The oxygen tank worked fine in isolation. But in the context of ground support equipment, in the context of the 65-volt upgrade, in the context of the testing procedures... it was a bomb waiting to go off.
>
> Good component engineering. Failed systems engineering. The tank designers focused on their component. Nobody looked at the whole system context. Nobody asked, 'What else changed? What are the interfaces? What are the hidden interactions?'
>
> That's systems engineering. And when you don't do it, this is what happens."

PPT Content:

```
SYSTEMS ENGINEERING LESSONS:

✗ Component worked in isolation
✗ Failed in system context
✗ Interface assumptions were wrong
✗ Design changes didn't propagate
✗ Testing didn't reveal system-level issue

CRITICAL SE PRACTICES MISSING:
• Interface control and verification
• Change impact analysis
• System-level requirements
• Context-aware testing
• Cross-functional communication

"Good parts do not guarantee a good system."
```

Learning Outcomes: LO1, LO3 - SE principles and practices

Slide 12: Case Study 2 - Therac-25: When Software Kills

Visual: Image of Therac-25 radiation therapy machine

Instructor Narration:

> "1985 to 1987. Six patients receiving radiation therapy. What should have been healing doses became lethal. Massive radiation overdoses—100 times the intended amount. Three patients died within months. Three more suffered severe injuries.

The machine? Therac-25. A computer-controlled radiation therapy device. And this case study should terrify every software engineer in this room. Because the root cause was software. More specifically, it was the *systems engineering* of software."

PPT Content:

```
THERAC-25 RADIATION OVERDOSES
1985-1987 | Six incidents

THE INCIDENTS:
• 6 patients received massive radiation overdoses
• 100x intended dosage
• 3 deaths, 3 severe injuries
• Machine continued operating normally
• Operators saw error codes, but no explanation

The Problem: Software failure + Systems engineering failure
```

Learning Outcomes: LO3 - Software-intensive system failure

Slide 13: Therac-25 - The Systems Engineering Catastrophe

Visual: Diagram showing removed hardware interlocks, software dependency

Instructor Narration:

"Previous versions—Therac-6 and Therac-20—had hardware interlocks. Physical safety mechanisms that prevented overdoses even if software failed. But Therac-25 removed those. Why? Cost savings. Space savings. And confidence in software.

The engineers made the software responsible for safety. But the software had race conditions, inadequate error handling, and cryptic error messages. Operators saw 'Malfunction 54' and were told to continue treatment.

Here's the systems engineering failure: **They removed redundant safety layers without understanding the system-level risks**. They optimized for cost, not safety. They trusted a single layer of defense."

PPT Content:

```
THE SYSTEMS ENGINEERING FAILURES:

1. Safety Architecture Degradation
   • Removed hardware safety interlocks
   • Relied solely on software for safety
   • No defense-in-depth strategy

2. Software Requirements Gap
   • No formal safety requirements for software
   • Race conditions in critical code
   • Inadequate error handling

3. Human-System Interface Failure
```

```
   • Cryptic error messages ("Malfunction 54")
   • Operators not trained on error meanings
   • No clear safety guidance

4. Inadequate Testing
   • Software tested in isolation
   • Not tested for race conditions under realistic timing
   • System-level safety scenarios not exercised

5. Lack of System Safety Analysis
   • No failure modes and effects analysis (FMEA)
   • No hazard analysis
   • Didn't understand system-level failure modes
```

Learning Outcomes: LO1, LO3 - SE safety practices

Slide 14: Therac-25 - The Systems Lesson

Visual: Layered safety model showing single point of failure vs. defense-in-depth

Instructor Narration:

> "The lesson here is about **system safety architecture**. In previous versions, if software failed, hardware caught it. There were layers of defense. Therac-25 had a single point of failure: the software. And it failed.
>
> This introduced the concept of **defense-in-depth** to medical device regulation. It changed how we think about software safety. And it's directly relevant to automotive systems today.
>
> Think about it: Your autonomous emergency braking system. Your lane-keeping assist. Your adaptive cruise control. All software-intensive. All safety-critical. Are you building Therac-25, or are you building defensible systems?"

PPT Content:

```
SYSTEMS ENGINEERING LESSONS:

✗ Single point of failure (software only)
✗ No redundant safety layers
✗ Inadequate hazard analysis
✗ Poor human-system interface
✗ Testing didn't cover system failure modes

CRITICAL SE PRACTICES MISSING:
• System safety analysis (FMEA, hazard analysis)
• Defense-in-depth architecture
• Safety requirements for software
• Human factors engineering
• System-level integration testing

"Removing redundancy without system analysis is gambling with lives."
```

```
AUTOMOTIVE RELEVANCE: Software-intensive safety systems today
```

Learning Outcomes: LO1, LO3, LO4 - SE safety for automotive context

---

Slide 15: Case Study 3 - Toyota Unintended Acceleration

Visual: Image of Toyota vehicle, recall headlines

Instructor Narration:

> "2009. Reports start flooding in: Toyota vehicles accelerating uncontrollably. Drivers pressing the brake, but the car keeps going. 89 people die. Toyota denies a problem, then recalls 9 million vehicles. First blaming floor mats, then the accelerator pedal, then... nothing definitive.
>
> NASA engineers spent months analyzing the software. The result? They couldn't prove the software caused it, but they couldn't prove it *didn't*. The system was too complex to fully verify.
>
> This is the most important case study for you as automotive engineers. Because this is *your* domain. And the lessons are profound."

PPT Content:

```
TOYOTA UNINTENDED ACCELERATION
2009-2011 | Massive recall crisis

THE INCIDENTS:
• 89 confirmed deaths
• Thousands of unintended acceleration reports
• 9 million vehicles recalled globally
• $3 billion in costs
• Congressional hearings
• NASA investigation

Initial Causes Blamed:
1. Floor mats
2. Sticky accelerator pedals
3. Software? (Never definitively proven)

The Real Problem: System complexity beyond verification
```

Learning Outcomes: LO3, LO4 - Automotive SE failure

Slide 16: Toyota - The Systems Engineering Breakdown

Visual: Complexity diagram showing ECU interactions, software interdependencies

Instructor Narration:

"Here's what NASA found: Toyota's Electronic Throttle Control System had 67 tasks running in a single CPU. Global variables everywhere. No memory protection. Thousands of potential race conditions. Stack overflow vulnerabilities.

Could it cause unintended acceleration? Maybe. Could NASA prove it? No. The system was too complex. And that's the systems engineering failure.

If you can't verify your system works, you don't have engineering—you have hope. Toyota built a system so complex that even NASA couldn't fully analyze it. They violated fundamental systems engineering principles: simplicity, modularity, verifiability."

PPT Content:

```
THE SYSTEMS ENGINEERING FAILURES:

1. Unmanageable Complexity
   • 67 tasks on single CPU
   • Global variables shared across functions
   • No memory protection between tasks
   • 11,000+ global variables in ETCS software

2. Inadequate Software Architecture
   • Spaghetti code structure
   • Tight coupling between functions
   • Poor modularity
   • Stack overflow vulnerabilities

3. Insufficient Verification
   • System too complex to fully verify
   • NASA: 94% code coverage, but still couldn't guarantee safety
   • Unable to prove absence of failure modes

4. Requirements Traceability Gap
   • Difficult to trace code to requirements
   • Changes propagated unpredictably
   • No clear system-level safety architecture

5. Suppressed Warnings & Cultural Issues
   • Safety concerns allegedly ignored
   • Pressure to meet cost/schedule over safety
   • Organizational culture didn't prioritize SE rigor
```

Learning Outcomes: LO1, LO3, LO4 - SE in automotive software

RISING ACTION: Why Systems Engineering Matters (Slides 17-22, ~20 minutes)

Narrative Voice: Analytical, extracting patterns, building understanding

---

Slide 17: Common Patterns Across All Three Failures

Visual: Venn diagram showing overlapping failure modes

Instructor Narration:

> "Step back and look at all three disasters. Different decades. Different technologies. Different industries. But notice the patterns:
>
> **All three had interface failures**—Apollo's voltage mismatch, Therac's software-hardware interface, Toyota's task interactions.
>
> **All three had complexity beyond verification**—Apollo's ground equipment interactions, Therac's timing-dependent race conditions, Toyota's unverifiable software.
>
> **All three had inadequate system-level thinking**—focused on components, missed system behaviors.
>
> These aren't random failures. These are *systems engineering* failures. And they follow predictable patterns."

PPT Content:

```
COMMON FAILURE PATTERNS:

1. INTERFACE FAILURES
   • Apollo: 28V vs 65V mismatch
   • Therac: Hardware-software safety interface
   • Toyota: Inter-task communication errors

2. EMERGENT COMPLEXITY
   • Apollo: Ground equipment interaction
   • Therac: Race conditions under timing scenarios
   • Toyota: Global variable interactions

3. VERIFICATION GAPS
   • Apollo: Ground compatibility not verified
   • Therac: Timing-dependent behaviors not tested
   • Toyota: Too complex to fully verify

4. MISSING SYSTEM-LEVEL REQUIREMENTS
   • Apollo: No ground equipment compatibility spec
   • Therac: No system safety architecture requirement
   • Toyota: No software complexity limits

→ These are SYSTEMS ENGINEERING failures, not component failures
```

Learning Outcomes: LO1, LO3 - Recognizing SE failure patterns

---

Slide 18: The Cost of Poor Systems Engineering

Visual: Financial and human cost infographic

Instructor Narration:

> "Let's talk about cost. Not just money, though that's significant. Let's talk about the full cost of poor systems engineering.
>
> **Financial**: Toyota: $3 billion. Mars Climate Orbiter (metric/imperial unit error): $125 million. Boeing 737 MAX: over $20 billion and counting.
>
> **Human**: Therac-25: 6 victims. Toyota: 89 deaths. Boeing 737 MAX: 346 deaths. Apollo 13: three astronauts nearly lost.
>
> **Reputation**: Toyota lost its reputation for quality. Boeing lost decades of safety credibility.
>
> But here's the key: **Every one of these was preventable with proper systems engineering**. These weren't acts of God. These were process failures."

PPT Content:

```
THE COST OF POOR SYSTEMS ENGINEERING

FINANCIAL COSTS:
• Toyota recalls: $3B+
• Mars Climate Orbiter: $125M (unit conversion error)
• Boeing 737 MAX: $20B+
• Average automotive recall: $500M-$1B

HUMAN COSTS:
• Therac-25: 6 casualties
• Toyota: 89 confirmed deaths
• Boeing 737 MAX: 346 deaths
• Countless injuries and trauma

REPUTATIONAL COSTS:
• Decades to rebuild trust
• Lost market share
• Regulatory scrutiny
• Industry-wide impacts

TIME COSTS:
• Years of investigation
• Development delays
• Rework and redesign
• Regulatory approval delays

→ ALL PREVENTABLE WITH RIGOROUS SYSTEMS ENGINEERING
```

Learning Outcomes: LO3 - Understanding SE value proposition

Slide 19: The ROI of Systems Engineering

Visual: Graph showing SE investment vs. total lifecycle cost

Instructor Narration:

> "Now let's flip it around. What's the return on investment for *good* systems engineering?
>
> Studies show that every dollar spent on early systems engineering saves **seven to ten dollars** later in development. Finding a requirements error before design? Costs $100. Finding it after production? Costs $10,000 to $1,000,000.
>
> More importantly: **systems engineering front-loads the work**. You spend more time upfront in requirements, architecture, planning. But you spend dramatically less time in rework, recalls, and firefighting.
>
> Think of it this way: Would you rather spend 3 months getting requirements right, or 3 years fixing a recalled product?"

PPT Content:

```
RETURN ON INVESTMENT: SYSTEMS ENGINEERING

COST MULTIPLIER OF FIXING ERRORS:
• Requirements phase: 1x (baseline)
• Design phase: 10x
• Implementation: 100x
• Testing: 1,000x
• Post-production: 10,000x+

SE INVESTMENT PAYOFF:
• $1 in early SE → Save $7-$10 later
• Reduce rework by 50-80%
• Cut integration time by 30-60%
• Prevent costly recalls and redesigns

FRONT-LOADING BENEFITS:
• More time in requirements → Less time in rework
• Better architecture → Easier integration
• Early risk identification → Fewer surprises
• Clear verification plan → Faster testing

"Pay now or pay more later. Systems engineering is insurance against disas
```

Learning Outcomes: LO3 - Economic value of SE

Slide 20: What Systems Engineering Prevents

Visual: Shield diagram showing SE protecting against various failure modes

Instructor Narration:

> "So what exactly does systems engineering prevent? Let me give you a practical list. This is what you get when you practice rigorous SE:
>
> **Interface mismatches** like Apollo 13—you define and verify all interfaces. **Scope creep**—you manage requirements changes systematically. **Integration nightmares**

—you plan integration from the start. **Unverifiable complexity**—you maintain system simplicity and modularity. **Missing requirements**—you engage stakeholders and capture all needs. **Safety hazards**—you analyze failure modes proactively.

Systems engineering is defensive engineering. It's how you prevent disasters before they happen."

PPT Content:

```
WHAT SYSTEMS ENGINEERING PREVENTS:

✓ Interface mismatches and incompatibilities
✓ Requirements creep and uncontrolled changes
✓ Integration failures and late surprises
✓ Unverifiable system complexity
✓ Missing critical requirements
✓ Safety hazards and failure modes
✓ Cost overruns from rework
✓ Schedule delays from firefighting
✓ Quality issues and field failures
✓ Regulatory non-compliance

HOW SE PREVENTS THESE:
• Structured requirements processes
• Interface control and verification
• System architecture and design
• Risk and failure mode analysis
• Configuration management
• Verification and validation planning
• Stakeholder engagement
• Lifecycle thinking

"Systems engineering is proactive, not reactive."
```

Learning Outcomes: LO1, LO3 - SE practices and value

Slide 21: Systems Engineering in Automotive Context

Visual: Modern vehicle architecture diagram showing system complexity

Instructor Narration:

"Let's bring this to your world. Modern vehicles are arguably the most complex consumer products ever built. You have:

Mechanical systems, electrical systems, software systems, embedded systems, communication networks, user interfaces, cloud connectivity, over-the-air updates, autonomous features, safety systems, comfort systems, infotainment...

And all of these must work together, in every climate, for 10+ years, in the hands of untrained users, while meeting safety standards and regulatory requirements.

Without systems engineering, this is impossible. Not difficult—impossible. You cannot build modern vehicles with component-focused engineering alone."

PPT Content:

```
AUTOMOTIVE SYSTEMS ENGINEERING CHALLENGES

SYSTEM COMPLEXITY:
• 100M+ lines of code (high-end vehicles)
• 150+ ECUs networked together
• 30,000+ components from 1,000+ suppliers
• Multiple domains: powertrain, chassis, body, ADAS, infotainment
• Safety-critical + non-critical systems integrated

REQUIREMENTS CHALLENGES:
• Safety (ISO 26262, FMVSS)
• Security (cybersecurity, data privacy)
• Performance (acceleration, handling, efficiency)
• Comfort and convenience
• Regulatory compliance (emissions, crash, etc.)
• Cost targets and manufacturability

LIFECYCLE CHALLENGES:
• 3-5 year development cycles
• 10-15 year operational life
• Extreme environmental conditions
• Untrained end users
• Over-the-air updates and evolution

→ SYSTEMS ENGINEERING IS NOT OPTIONAL IN AUTOMOTIVE
```

Learning Outcomes: LO3, LO4 - SE in automotive context

Slide 22: The Systems Engineering Imperative

Visual: Powerful visual showing SE as the foundation

Instructor Narration:

"Here's the bottom line. In automotive engineering today, you have two choices:

Choice 1: Practice rigorous systems engineering. Define requirements. Design architectures. Manage interfaces. Verify systematically. It's more work upfront. It requires discipline. It requires thinking beyond your component.

Choice 2: Skip systems engineering. Move fast. Focus on your piece. Hope it all works when integrated. Then spend years in recalls, lawsuits, and reputation damage. If you're lucky.

There is no middle ground. You either do systems engineering, or you pay the price. The case studies prove it.

So the question isn't *whether* to do systems engineering. It's *how* to do it well. That's what this course teaches."

PPT Content:

```
THE SYSTEMS ENGINEERING IMPERATIVE

TWO PATHS:

PATH 1: Rigorous Systems Engineering
→ Front-loaded effort
→ Disciplined processes
→ System-level thinking
→ Reduced risk
→ Successful products
→ Professional reputation

PATH 2: Skip Systems Engineering
→ Fast start, slow finish
→ Ad-hoc processes
→ Component-focused
→ Hidden risks emerge late
→ Recalls and failures
→ Career consequences

VERDICT:
"Systems engineering is not optional for complex systems.
The only question is how well you do it."

This course teaches you HOW.
```

Learning Outcomes: LO1, LO3 - SE value proposition and commitment

---

CLIMAX: What IS a System? (Slides 23-28, ~25 minutes)

Narrative Voice: Definitional, building foundational concepts clearly

---

Slide 23: Defining "System" - First Principles

Visual: Simple diagram showing elements interconnected

Instructor Narration:

> "We've talked about systems engineering without defining 'system.' Let's fix that. What *is* a system?
>
> Here's the formal definition from INCOSE—the International Council on Systems Engineering: **'A system is a combination of interacting elements organized to achieve one or more stated purposes.'**
>
> Let me break that down, because every word matters."

PPT Content:

```
WHAT IS A SYSTEM?

INCOSE DEFINITION:
"A system is a combination of interacting elements
organized to achieve one or more stated purposes."

KEY WORDS:
• COMBINATION → Multiple parts
• INTERACTING → Not just collection, but connections
• ELEMENTS → Components, subsystems, people, processes
• ORGANIZED → Structured, not random
• ACHIEVE → Goal-oriented
• STATED PURPOSES → Defined objectives/functions

Examples:
✓ Vehicle (interacting subsystems to provide transportation)
✓ Hospital (people, equipment, processes to provide healthcare)
✓ Supply chain (organizations, logistics to deliver products)
✗ Pile of parts (no organization, no purpose)
```

Learning Outcomes: LO1 - Understanding system definition

Slide 24: System Characteristics - What Makes It a System?

Visual: Diagram illustrating each characteristic

Instructor Narration:

> "A system has specific characteristics that distinguish it from just a random collection of things. Let me give you six key characteristics:
>
> **1. Purpose**: Every system exists to achieve specific objectives. A vehicle's purpose is transportation.
>
> **2. Boundaries**: Systems have defined limits. What's inside the system? What's outside? For a vehicle, is the driver inside or outside the system boundary? It depends on your perspective.
>
> **3. Elements**: The components and subsystems. Engine, transmission, wheels, software, etc.
>
> **4. Interactions**: How elements work together. This is where complexity lives.
>
> **5. Emergent Properties**: The system has properties that individual elements don't have. A wheel can't transport you. An engine can't transport you. But a *vehicle system* can.
>
> **6. Environment**: Systems operate in a context. Weather, roads, regulations, user behavior—all external factors affecting the system.
>
> Understanding these characteristics is fundamental to systems thinking."

PPT Content:

```
SIX KEY SYSTEM CHARACTERISTICS

1. PURPOSE / FUNCTION
   • Defined objectives and goals
   • Reason for existence
   • Measurable outcomes

2. BOUNDARIES
   • What's IN the system vs. OUT
   • Defines scope
   • Can be physical, functional, or organizational

3. ELEMENTS / COMPONENTS
   • Individual parts and subsystems
   • Can be hardware, software, people, processes
   • Hierarchical structure

4. INTERFACES / INTERACTIONS
   • How elements connect and communicate
   • Energy, data, material flows
   • Dependencies and relationships

5. EMERGENT PROPERTIES
   • System-level behaviors not present in individual elements
   • "The whole is greater than the sum of parts"
   • Cannot be predicted from component analysis alone

6. ENVIRONMENT / CONTEXT
   • External factors affecting the system
   • Operating conditions
   • Stakeholders and external systems

→ All six must be considered in systems engineering
```

Learning Outcomes: LO1, LO3 - System characteristics

Slide 25: Emergent Properties - The Key Insight

Visual: Illustration showing individual parts vs. system behavior

Instructor Narration:

> "Let me emphasize emergent properties because this is the *key* systems engineering insight.
>
> **Emergence means the system exhibits behaviors that you cannot predict by analyzing individual components alone.** An airplane's ability to fly is an emergent property. No single component flies. But the *system* flies.

In automotive: vehicle handling is emergent. No single component 'handles.' But suspension + tires + steering + chassis + weight distribution creates handling behavior.

This is why component-focused engineering isn't enough. **You can optimize every component and still have a terrible system** if you don't understand emergent properties. That's systems thinking."

PPT Content:

```
EMERGENT PROPERTIES: The Whole > Sum of Parts

DEFINITION:
Properties and behaviors that arise from interactions,
not predictable from individual element analysis

AUTOMOTIVE EXAMPLES:

Component Level → System Level (Emergent)
• Tire grip + suspension + steering → Vehicle handling
• Engine + transmission + wheels → Vehicle performance
• Sensors + software + actuators → Autonomous behavior
• Individual ECUs + network → System timing behaviors

KEY INSIGHTS:
✓ Cannot be designed at component level alone
✓ Emerge from interfaces and interactions
✓ Often unpredictable (good or bad)
✓ Require system-level analysis and testing
✓ Source of both innovation and failure

Examples of Negative Emergence:
• Toyota unintended acceleration (interaction of multiple systems)
• Therac-25 race conditions (timing-dependent behavior)

"You cannot understand a system by analyzing parts in isolation."
```

Learning Outcomes: LO1, LO3 - Understanding emergence and systems thinking

---

Slide 26: System Boundaries - Defining Your Scope

Visual: Diagram showing same vehicle with different system boundaries

Instructor Narration:

"System boundaries are trickier than they seem. Let me show you. Is this vehicle a system? Yes. But what's the boundary?

Perspective 1: The vehicle itself. Driver is external. Road is external.

Perspective 2: The vehicle + driver. Now human factors are internal. Driver errors are system failures.

Perspective 3: Vehicle + driver + road + traffic. Now you're looking at a transportation system.

There's no single 'right' boundary. It depends on your purpose, your requirements, your stakeholder perspective. But you *must* define it clearly. Because what's inside your boundary is what you're responsible for engineering.

Boundary confusion causes requirements gaps and interface failures. Apollo 13? Boundary confusion between spacecraft systems and ground systems."

PPT Content:

```
SYSTEM BOUNDARIES: Defining Your Scope

SAME VEHICLE, DIFFERENT BOUNDARIES:

Boundary 1: Vehicle Only
• Focus: Vehicle subsystems
• Driver: External user
• Environment: External factors
• Responsibilities: Vehicle functions

Boundary 2: Vehicle + Driver
• Focus: Human-vehicle system
• Driver: Part of system (human factors critical)
• Environment: External
• Responsibilities: Include usability, ergonomics

Boundary 3: Vehicle + Driver + Infrastructure
• Focus: Transportation system
• Roads, traffic, V2X: All internal
• Society: External
• Responsibilities: System-of-systems integration

KEY PRINCIPLES:
✓ Boundary depends on perspective and purpose
✓ Must be explicitly defined
✓ Everything inside = your responsibility
✓ Interfaces at boundary = critical
✓ Boundary errors cause requirements gaps

"Define your boundary or your boundary will define you."
```

Learning Outcomes: LO1, LO3 - System boundary definition

Slide 27: System Hierarchy - Systems of Systems

Visual: Nested hierarchy diagram from component to system-of-systems

Instructor Narration:

"Systems exist at multiple levels. This is called **system hierarchy**. Let me show you for a vehicle:

Component Level: A fuel injector, a sensor, a microcontroller.

Subsystem Level: The fuel system, the braking system, the infotainment system.

System Level: The complete vehicle.

System-of-Systems Level: The vehicle + smart infrastructure + traffic management + other vehicles.

Here's what's critical: **At each level, you need systems engineering**. The fuel system needs SE. The vehicle needs SE. The transportation system needs SE.

And here's the challenge: You need to think at multiple levels simultaneously. Your fuel injector is a component to you, but it's a system to the supplier who builds it."

PPT Content:

```
SYSTEM HIERARCHY: Nested Levels

LEVEL 1: COMPONENT
• Individual parts (sensor, actuator, ECU chip)
• Defined function, interfaces
• Example: ABS wheel speed sensor

LEVEL 2: SUBSYSTEM
• Collection of components with specific function
• Example: Complete ABS system (sensors, hydraulic unit, ECU)

LEVEL 3: SYSTEM
• Complete product delivering end-user value
• Example: Complete vehicle

LEVEL 4: SYSTEM-OF-SYSTEMS (SoS)
• Multiple independent systems working together
• Example: Connected vehicles + smart infrastructure + traffic management

KEY INSIGHTS:
✓ Systems engineering applies at every level
✓ One person's system is another's component
✓ Must manage interfaces across levels
✓ Requirements flow down, verification flows up
✓ Complexity increases at higher levels

Automotive Example - Braking System:
Component: Brake pad
↓
Subsystem: Disc brake assembly
↓
System: Complete braking system (ABS, ESC, brake-by-wire)
↓
SoS: Vehicle-to-vehicle cooperative braking
```

Learning Outcomes: LO1, LO3 - System hierarchy and complexity

---

Slide 28: System Interfaces - Where Failure Lives

Visual: Interface diagram showing different types of interfaces

Instructor Narration:

> "We've talked about interfaces in every failure case study. Let me be direct: **Most system failures happen at interfaces, not within components.**
>
> Apollo 13? Interface between ground equipment and spacecraft. Therac-25? Interface between hardware and software safety. Toyota? Interface between software tasks and shared memory.
>
> Interfaces are where different teams meet. Different suppliers. Different disciplines. Different assumptions. That's where mismatches hide.
>
> Systems engineering focuses intensely on interfaces: defining them, documenting them, verifying them, controlling changes to them. Master interfaces, and you prevent most disasters."

PPT Content:

```
SYSTEM INTERFACES: Where Failure Lives

TYPES OF INTERFACES:

1. PHYSICAL / MECHANICAL
   • Connectors, mounting points
   • Example: ECU connector pinout

2. ELECTRICAL / POWER
   • Voltage levels, current capacity, grounding
   • Example: 12V vs 48V systems

3. DATA / COMMUNICATION
   • Protocols, data formats, timing
   • Example: CAN bus message definitions

4. FUNCTIONAL / LOGICAL
   • Input/output relationships, behaviors
   • Example: Brake signal → ABS response

5. ENVIRONMENTAL
   • Temperature, vibration, EMI
   • Example: Underhood environment exposure

6. HUMAN-SYSTEM
   • Controls, displays, user expectations
   • Example: Pedal force vs vehicle response

INTERFACE MANAGEMENT:
✓ Define all interfaces explicitly
✓ Document interface control documents (ICDs)
✓ Verify interface compatibility
```

```
✓ Control interface changes
✓ Test interface interactions

"80% of system failures occur at interfaces, not within components."

→ Systems engineering is interface engineering
```

Learning Outcomes: LO1, LO3 - Interface management criticality

---

RESOLUTION: Systems Thinking & Moving Forward (Slides 29-30, ~20 minutes)

Narrative Voice: Inspirational, forward-looking, empowering

Slide 29: Introduction to Systems Thinking

Visual: Comparison table showing traditional vs systems thinking

Instructor Narration:

> "We've covered a lot today. Now I want to introduce you to *systems thinking*—the mindset that underlies all of systems engineering.
>
> **Traditional thinking** focuses on isolated components, linear cause-effect, optimization of parts, and reductionism—break it down to understand it.
>
> **Systems thinking** focuses on relationships and interactions, feedback loops and dynamics, optimization of the whole system, and emergence—understand the whole to understand behavior.
>
> Let me give you examples. Traditional thinking: 'The engine needs more power.' Systems thinking: 'What's the vehicle-level performance requirement? Is more engine power the right solution, or can we reduce weight, improve aerodynamics, or optimize the transmission?'
>
> Traditional: 'This software module crashed.' Systems: 'What interactions with other modules caused the crash? What's the system-level failure mode?'
>
> This is the mindset shift you need to make. From parts to wholes. From isolation to context. From components to systems."

PPT Content:

```
SYSTEMS THINKING: The Essential Mindset

TRADITIONAL THINKING vs SYSTEMS THINKING

Focus:
Traditional: Individual components
Systems: Relationships and interactions

Causality:
```

```
Traditional: Linear cause → effect
Systems: Feedback loops, circular causality

Optimization:
Traditional: Optimize each part
Systems: Optimize the whole (may sub-optimize parts)

Analysis:
Traditional: Reductionism (break down)
Systems: Holism (understand whole context)

Problem-Solving:
Traditional: Fix the failing component
Systems: Understand system dynamics causing failure

SYSTEMS THINKING PRINCIPLES:

1. The whole is greater than sum of parts
2. Understand relationships, not just elements
3. Consider context and environment
4. Recognize feedback loops and dynamics
5. Expect emergent behaviors
6. Think in hierarchies and levels
7. Anticipate unintended consequences
8. Balance short-term and long-term effects

AUTOMOTIVE EXAMPLE:
Traditional: "Add more airbags for safety"
Systems: "How does the entire safety system (structure, restraints,
ADAS, driver behavior) work together? Where's the optimal investment?"

→ Systems engineering applies systems thinking systematically
```

Learning Outcomes: LO3 - Systems thinking mindset

Slide 30: Your Journey Ahead - Course Roadmap

Visual: Course module overview showing progression

Instructor Narration:

> "Today was your wake-up call. You've seen why systems engineering matters through spectacular failures. You've learned what a system is and its characteristics. You've begun developing systems thinking.
>
> Here's where we go next:
>
> **Module 1 (Sessions 1-4):** We'll build your foundation—systems engineering fundamentals, MBSE, requirements engineering. The core knowledge.
>
> **Module 2 (Sessions 5-8):** System design, analysis, integration, verification, and validation. The technical processes.

Module 3 (Sessions 9-12): Project management, risk, decision-making, quality. The management practices.

Module 4 (Sessions 13-16): Life cycle cost, emerging trends, putting it all together.

By the end, you won't just understand systems engineering—you'll be able to *practice* it on real automotive systems. That's the goal."

PPT Content:

```
YOUR JOURNEY: Course Roadmap

SESSION 1 (Today): Introduction - WHY systems engineering matters

MODULE 1: Foundations (Sessions 1-4)
✓ Session 1: Introduction & failures
→ Session 2: SE fundamentals, systems thinking, INCOSE framework
→ Session 3: Traditional vs MBSE
→ Session 4: Requirements engineering

MODULE 2: System Design & Analysis (Sessions 5-8)
→ System design and architecture
→ Case Study 1
→ Operational and functional analysis
→ Integration, verification & validation

MODULE 3: Technical & Project Management (Sessions 9-12)
→ Project and risk management
→ Decision and quality management
→ Agreement and infrastructure management
→ Specialty engineering: Safety & security

MODULE 4: Lifecycle & Emerging Trends (Sessions 13-16)
→ Life cycle cost and TCO
→ Case Study 2
→ Technical management processes
→ Emerging trends (AI, digital engineering, autonomous systems)

NEXT SESSION: Systems Engineering Fundamentals
• INCOSE definition and framework
• ISO/IEC 15288 lifecycle processes
• V-model
• Stakeholder engagement
• Systems thinking deep dive
```

Learning Outcomes: Course overview, LO1

Slide 31: Key Takeaways - What You Learned Today

Visual: Clean summary with key points

Instructor Narration:

"Let me summarize what you learned today. Write these down:

1. Systems engineering prevents disasters. Apollo 13, Therac-25, Toyota—all preventable with rigorous SE.

2. The cost of poor SE is enormous—financially, human lives, reputation. The ROI of good SE is 7-10x.

3. A system is interacting elements organized for a purpose. It has boundaries, elements, interfaces, emergent properties, and operates in an environment.

4. Most failures happen at interfaces, not within components. SE focuses on interfaces.

5. Systems thinking is essential—think about the whole, not just parts. Consider interactions, emergence, context.

6. In modern automotive engineering, SE is not optional. Complexity demands it.

This is your foundation. We build from here."

PPT Content:

```
KEY TAKEAWAYS - SESSION 1

1. SYSTEMS ENGINEERING PREVENTS DISASTERS
   • Case studies prove the consequences of poor SE
   • All failures were preventable with rigorous processes

2. SE HAS COMPELLING ROI
   • $1 in early SE → Save $7-$10 later
   • Prevents recalls, rework, reputation damage
   • Front-loads effort, reduces late-stage chaos

3. SYSTEM DEFINITION & CHARACTERISTICS
   • Interacting elements organized for a purpose
   • Six characteristics: purpose, boundaries, elements,
     interactions, emergent properties, environment

4. INTERFACES ARE CRITICAL
   • 80% of failures occur at interfaces
   • Interface management is core SE practice

5. SYSTEMS THINKING MINDSET
   • Focus on whole, not just parts
   • Understand interactions and emergence
   • Consider context and environment

6. AUTOMOTIVE SE IS ESSENTIAL
   • Complexity of modern vehicles demands SE
   • Safety-critical nature leaves no room for error
   • SE is a professional responsibility

→ You are beginning your journey from component engineer to systems engine
```

Learning Outcomes: LO1, LO3

Slide 32: Reflection Questions

Visual: Thought-provoking questions

Instructor Narration:

> "Before we go to Q&A, I want you to think about these questions. You don't have to answer now, but consider them:
>
> **1.** Think about a project you've worked on. What interfaces did you assume would work? Did you verify them?
>
> **2.** In your current or past work, have you seen examples of component-focused thinking causing system-level problems?
>
> **3.** If you were designing a safety-critical automotive system today, what lessons from Therac-25 and Toyota would you apply?
>
> **4.** What's harder: designing a good component, or integrating components into a good system? Why?
>
> Systems engineering is about asking better questions. Start asking them."

PPT Content:

```
REFLECTION QUESTIONS

Consider these as you move forward:

1. PERSONAL EXPERIENCE
   • What interfaces have you assumed would work?
   • Did you verify them, or hope for the best?

2. COMPONENT VS SYSTEM THINKING
   • Have you seen component-focused thinking cause system problems?
   • What were the consequences?

3. APPLYING LESSONS LEARNED
   • You're designing a safety-critical automotive system
   • What lessons from today's case studies would you apply?
   • What SE practices would you prioritize?

4. ENGINEERING PHILOSOPHY
   • What's harder: designing a component or integrating a system?
   • Why is systems engineering a different discipline?

5. FUTURE THINKING
   • As vehicles become more automated and connected...
   • How will SE become even more critical?
   • What new challenges will emerge?
```

```
"Systems engineering is asking better questions."
```

Learning Outcomes: LO3 - Reflection and application

Slide 33: Assignment & Preparation for Next Session

Visual: Assignment details

Instructor Narration:

> "Here's your assignment for next session:
>
> **Part 1:** Read INCOSE Systems Engineering Handbook, Chapter 1. Focus on the definition of systems engineering and the key concepts.
>
> **Part 2:** Find one example of a systems engineering failure from the last 5 years. It can be automotive or any other industry. Write a one-page analysis: What happened? What SE practices were missing? How could it have been prevented?
>
> **Part 3:** Think about a complex product you use daily (phone, laptop, car). Map out its system boundaries, major subsystems, and at least five critical interfaces. Bring this to class—we'll discuss.
>
> Next session, we dive deep into systems engineering fundamentals—the INCOSE framework, ISO/IEC 15288 processes, the V-model, and stakeholder engagement. This is where we build your systematic SE knowledge."

PPT Content:

```
ASSIGNMENT - Due Before Session 2

PART 1: Reading
• INCOSE SE Handbook, Chapter 1
• Focus: SE definition, key concepts, SE processes overview
• Be prepared to discuss

PART 2: Case Study Analysis (1 page)
Find a systems engineering failure (last 5 years)
Analyze:
• What happened? (brief description)
• What SE practices were missing?
• How could it have been prevented?
• What lessons are relevant to automotive SE?

PART 3: System Mapping Exercise
Choose a complex product you use daily
Map:
• System boundaries (what's in, what's out)
• Major subsystems (at least 5)
• Critical interfaces (at least 5)
• One emergent property
```

```
Bring to Session 2 for discussion

NEXT SESSION PREVIEW:
Session 2: Systems Engineering Fundamentals
• INCOSE definition and principles
• ISO/IEC 15288 life cycle processes
• System life cycle stages
• The V-model and variations
• Stakeholder engagement
• Deep dive into systems thinking
```

Learning Outcomes: LO1, LO3 - Preparation and practice

Slide 34: Q&A - Discussion Topics

Visual: Guiding questions for discussion

Instructor Narration:

> "We have about 20-30 minutes for questions and discussion. Here are some topics we can explore if you don't have specific questions:
>
> • Deep dive into any of the case studies
> • How SE applies to your specific background or interests
> • The relationship between SE and your discipline (software, mechanical, electrical)
> • Current automotive SE challenges (autonomous vehicles, electrification, connectivity)
> • How to start practicing SE in your current work or projects
>
> What questions do you have?"

PPT Content:

```
Q&A / DISCUSSION

Open Questions:

1. CASE STUDIES
   • Want more detail on Apollo 13, Therac-25, or Toyota?
   • Other failure examples you're curious about?

2. AUTOMOTIVE APPLICATIONS
   • How does SE apply to autonomous vehicles?
   • SE in electric vehicle development?
   • Software-defined vehicles and SE?

3. DISCIPLINE INTEGRATION
   • How does SE relate to software engineering?
   • Mechanical engineering perspective?
   • Electrical/electronics engineering?

4. PRACTICAL APPLICATION
```

```
   • How to start practicing SE in your work?
   • SE in small teams vs large organizations?
   • SE tools and techniques preview?

5. CAREER & INDUSTRY
   • Systems engineer roles in automotive?
   • Industry trends in SE?
   • Skills to develop?

What questions do you have?
```

Learning Outcomes: LO1, LO3 - Discussion and deeper understanding

---

📊 PPT DESIGN GUIDANCE

Visual Style:

- **Professional engineering aesthetic**: Clean, modern layouts with automotive industry feel
- **Color scheme**:
- Primary: Deep blue (trust, engineering)
- Secondary: Silver/gray (automotive, technology)
- Accent: Orange/red for warnings, critical points
- Green for success, SE benefits
- **Typography**: Sans-serif, highly readable (Calibri, Arial, or Helvetica)
- **Consistent template** throughout with clear slide numbering

Key Slides to Emphasize:

1. **Slide 2** (The Question) - Hook with complexity visual
2. **Slides 9-16** (Case Studies) - Compelling visuals, impactful
3. **Slide 18** (Cost of Poor SE) - Infographic style, memorable
4. **Slide 23** (System Definition) - Crystal clear, foundational
5. **Slide 28** (Interfaces) - Diagram showing interface criticality
6. **Slide 31** (Key Takeaways) - Students will photograph this

Animations:

- **Minimal and professional** - avoid distracting animations
- Use **build animations** for:
- Multi-step case study timelines
- Revealing system characteristics one at a time
- Progressive disclosure of complex diagrams
- **Highlight/emphasis** for key terms and equations
- **Fade transitions** between major sections

Visual Elements:

- **Case study slides**: Powerful images (actual equipment/vehicles)
- **System diagrams**: Clean, professional technical diagrams
- **Cost graphics**: Infographics with clear visual hierarchy
- **Comparison slides**: Side-by-side layouts for traditional vs systems thinking

- **Use icons** for characteristics, principles (consistent style)

Technical Diagrams:

- Clear **system boundary diagrams** with dotted lines for boundaries
- **Interface diagrams** showing connections between elements
- **Hierarchy diagrams** showing nested system levels
- **Free-body style diagrams** for emergent properties
- Always include **labels, legends, and keys**

🎯 INSTRUCTOR NOTES

Pedagogical Strategy:

This session uses **Story-Based Learning with Case Studies** to:

1. **Create urgency and relevance**: Start with dramatic failures to answer "why should I care?"
2. **Emotional engagement**: Real disasters with real consequences create lasting memory
3. **Pattern recognition**: Three diverse case studies reveal common SE failure patterns
4. **Build from concrete to abstract**: Stories → Lessons → Concepts → Thinking
5. **Scaffold learning**: Move from motivation → definition → characteristics → mindset

Timing Breakdown (120 minutes total):

- **Setup (Slides 1-8)**: 20 minutes - Hook and context
- **Trigger (Slides 9-16)**: 25 minutes - Three case studies (8-9 min each)
- **Rising Action (Slides 17-22)**: 20 minutes - Why SE matters, cost/value
- **Climax (Slides 23-28)**: 25 minutes - System definition and characteristics
- **Resolution (Slides 29-34)**: 20 minutes - Systems thinking, takeaways, assignment
- **Q&A**: 10 minutes flexible (can extend to 30 minutes)

Common Student Questions/Challenges:

Q: "Isn't this just good engineering practice?" A: Yes, but it's *systematic* good practice. SE provides the framework, processes, and discipline to ensure good practices actually happen, especially in large, complex projects.

Q: "We don't have time for all this SE process." A: That's what Toyota thought. You don't have time NOT to do it. The time spent upfront saves multiples later. Show the ROI data again.

Q: "How is SE different from project management?" A: PM focuses on resources, schedule, budget. SE focuses on technical content—requirements, architecture, interfaces, verification. They complement each other.

Q: "Do we really need SE for smaller, simpler systems?" A: The rigor scales with complexity. Simple systems need less formal SE, but the thinking and key practices (requirements, interfaces, verification) always apply.

Engagement Strategies:

1. **During case studies (Slides 9-16)**:

2. Pause and ask: "What would you have done?"
3. Before revealing the SE failure, ask: "What do you think went wrong?"

4. Connect to students' experiences: "Have you seen similar issues?"

5. **During system definition (Slides 23-28)**:

6. Use think-pair-share: Give 2 minutes to identify system boundaries for their phone
7. Ask for examples of emergent properties they've experienced

8. Draw system hierarchy on board for a familiar product

9. **During systems thinking (Slide 29)**:

10. Present a simple problem, ask for traditional solution, then systems solution
11. Example: "Car has poor fuel economy. Traditional: bigger fuel tank. Systems: analyze whole energy system—engine efficiency, weight, aerodynamics, driver behavior"

Flexibility Options:

If running SHORT (need to finish in 100 minutes): - Reduce one case study detail (suggest Therac-25, keep Apollo 13 and Toyota) - Shorten Q&A section - Move reflection questions to assignment

If running LONG (students engaged, have 140 minutes): - Expand case study discussions with more technical details - Add breakout discussions during system characteristics - Include additional automotive failure examples - Extended Q&A session

Core Content (Must Cover): - All three case studies (at least summarized) - Cost/ROI of SE - System definition and six characteristics - Emergent properties concept - Interfaces criticality - Introduction to systems thinking

Storytelling Tips:

1. **Case Studies**: Tell them as *stories*, not reports
2. Use present tense for immediacy: "The astronauts *hear* a bang..."
3. Include human elements: fear, confusion, decisions under pressure
4. Build tension before revealing the SE failure

5. Show consequences vividly

6. **Personal Connections**:

7. Share any relevant personal industry experience
8. Ask students about their experiences with complex systems

9. Reference current events (recent recalls, tech failures)

10. **Use Analogies**:

11. System boundaries: Like defining a neighborhood vs a city vs a region
12. Emergent properties: Like individual musicians vs an orchestra
13. Interfaces: Like border crossings between countries—that's where issues happen

Assessment Checkpoints:

During Session: - After case studies: "What's the common pattern?" (should identify interface/complexity issues) - After system definition: "Is [example X] a system? Why or why not?" - After characteristics: "Name three characteristics of [familiar product]"

Assignment Success Criteria: - Part 2 case study should identify specific SE practices (requirements, interfaces, verification, etc.) - Part 3 system mapping should show understanding of boundaries and subsystems - Look for systems thinking language, not just component listing

---

✅ Session Success Criteria

Students leave able to: - ✓ Explain why systems engineering matters through case study examples - ✓ Articulate the cost of poor SE and ROI of good SE - ✓ Define "system" and identify six key system characteristics - ✓ Recognize interfaces as critical failure points - ✓ Distinguish between component thinking and systems thinking - ✓ Understand that SE is essential for modern automotive engineering

Most importantly: Students are **motivated** to learn systems engineering. They see it not as bureaucracy, but as essential professional practice. They understand that SE skills will make them better engineers and prevent disasters.

Mindset Shift Achieved:

From: "Systems engineering is overhead documentation" To: "Systems engineering is how we prevent disasters and build successful complex systems"

🔗 Bridge to Session 2

Setup for Next Session:

> "Today you learned *why* systems engineering matters and *what* a system is. Next session, we go deeper: *how* do we actually practice systems engineering? We'll explore the INCOSE framework, the ISO/IEC 15288 standard, system life cycle stages, the V-model, and stakeholder engagement. You'll learn the systematic processes and methods that make SE work. See you next time."

End of Framework

Framework Summary: - **Total Slides:** 34 (will condense to 28-30 for actual PPT with combined content) - **Story Arc:** Complete - Setup → Trigger (failures) → Rising Action (why SE matters) → Climax (what is a system) → Resolution (systems thinking) - **Engagement:** High - Case studies, discussions, reflection questions - **Learning Outcomes:** LO1, LO3 thoroughly addressed - **Automotive Context:** Strong - Toyota case study, modern vehicle complexity, safety criticality

Next Step: Instructor uses this framework to develop PowerPoint slides, inserting appropriate visuals, refining narration for personal style, and adapting timing to actual classroom dynamics.